# Memorystore - Michaël Bettan

| **Definition**: Reduce latency with scalable, secure, and highly available in-memory managed service for Redis and Memcached | **Availability**: Regional | **SLA**: 99.9% |
|---|---|---|

**Use Cases**: Caching data (very low-latency storage mechanism)

- ***Front-end for Databases***
- ***Online Games & Social Media****: Session Store, Content Caching, Jobs & Queues, Distributed Lock, Fast Data Ingestion, Stream Processing*
- ***Stream Processing (IoT) & Analytics****: Real-time Analytics, Leaderboard, Recommendation Engine, Counters, Page ranking*
- *Low-latency data access*
- *Fast read/write speeds for real-time dashboards*

**Billing**:

- **Service tier**: basic or standalone (HA) for Redis
- **Provisioned capacity**: how much storage and throughput is available.
- **Region**: location where the instance is provisioned.
- **Replicas (public preview)**: how many nodes are there in a Standard Tier instance.
- **vCPUs / cores** per node for Memcached
- **Memory provisioned** per node for Memcached

## Key capabilities

- 2 options: **Redis** & **Memcached**
- ***Scalable & HA:*** *replicated across zones*
- ***Deep Insights****: monitor instances using Cloud Operations*
- ***Backup data****: Easily backup instance data or import data into Memorystore from GCS buckets using RDB files*
- ***Interoperability:*** *no code change required*

## Ecosystem & Integration

- Open-source projects
- High-Availability made it easy
- Scale as needed (up to 30GiB Storage, 12 GPS throughput)
- Integration with
  - App Engine Standard
  - App Engine Flexible
  - Google Compute Engine (GCE)
  - Cloud Functions
  - Google Kubernetes Engine (GKE)
- Redis: Basic and Standard Tiers determine availability, cost, and performance.
  - **Basic**: Lower cost. Does not provide high availability.
  - **Standard**: Supports automatic failover for high availability and up to 5 read replicas for scaling reads.

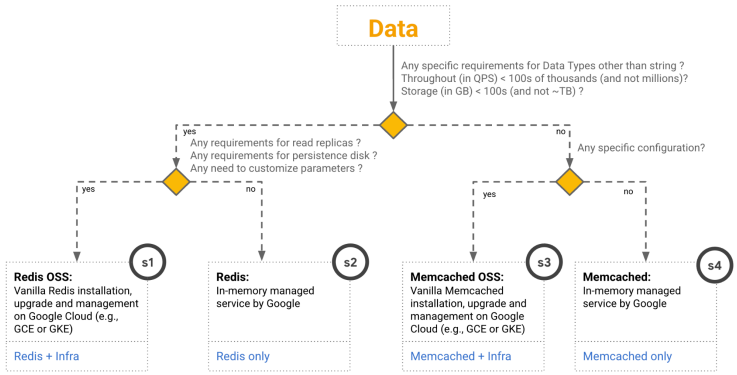
## Key considerations

- Memorystore does not support persistence, so it should not be used for applications that need to save data to persistent storage (RDB Snapshot in Preview).
- Even though it's a managed service, Monitor following:
  - Memory usage
  - # of expirable keys
  - Duration periods of memory overload
  - Cache-hit ratio

## Memorystore Redis vs. OSS Redis

- Persisting to Disk (Preview)
- Parameters are not customizable
- Lack of redis commands
- Standard Tier does not provide read from replica (Preview)

## Redis vs Memcached




| Feature | Redis | Memcached |
|---|---|---|
| Overall | Powerful data types & commands | Great for multi-thread and simplicity |
| Data Types | Strings, list, sets, sorted set, hashes, bit arrays, geospatial and hyper logs | strings |
| Memory Limit | 300 GB | 5,120 GB<br>(256 GB x 20 nodes) |
| Data size limits | 512 MB (string type) | 1 MB |
| Latency | Sub-milliseconds | Sub-milliseconds<br>Faster in general |
| Distributed caching cluster | Yes | No |
| Multi-thread | Yes | Yes |
| Scaling | Horizontal | Vertical |
| Replication | Yes | No |
| Transaction | Yes | No |
| Data Persistence | No (as of GA) | No |

## Optimizing Memorystore for Read Scalability

### Clustering

Memorystore for Redis clusters distribute data across multiple nodes, improving read scalability.

- **Cluster Size:**
  - Choose based on anticipated read load.
  - Start with a small cluster (e.g., 3 nodes) and scale up as needed.
  - Each additional node increases read capacity linearly.

- **Node Types:**
  - **Primary Node:** Handles writes and reads.
  - **Replica Nodes:** Handle reads only, improving read scalability.
- **Cluster Modes:**
  - **Standard Mode:** Single primary node with multiple read replicas.
  - **Cluster Mode:** Multiple primary nodes (shards) with their own replicas.
    - Offers better scalability for large datasets and high-throughput scenarios.

### Optimizing for Read Scalability

- **Connection Pooling:** Efficiently managing **client connections** is crucial. Incorrectly managing connections can lead to bottlenecks even with a large cluster.
- **Read Replicas:** While Memorystore doesn't offer **read replicas** in the same way some database systems do, understanding how to effectively distribute read traffic across multiple nodes within a cluster is vital for scalability.
- **Data Partitioning (Sharding):** If the dataset is very large, you might need to consider how to **distribute data across different Redis instances** (either within a cluster or across multiple clusters) to improve read performance. This involves understanding key strategies for partitioning data based on consistent hashing or other techniques.
- **Caching Strategies**: Understanding how to effectively use **Redis's data structures** (lists, sets, hashes) and commands to optimize reads is crucial.

### Disaster Recovery

- **Production vs. Development:** Always test failover and scaling in a dedicated development environment to avoid impacting production data and traffic.
- **force-data-loss** mode employs a chain of failover strategies to aggressively execute the failover. It does not check the offset delta between the primary and replicas before initiating the failover; you can potentially lose more than 30MB of data changes → **testing only**.
- **limited-data-loss** mode **minimizes data loss** by verifying that the difference in data between the primary and replica is below 30 MB before initiating the failover. The offset on the primary is incremented for each byte of data that must be synchronized to its replicas → **not a complete test**